American International University Bangladesh

Faculty of Science and Technology

Advanced Database Management System

Project Report on

Sports Tournament Management System

Section C

Group 04

Submitted by

SL	NAME	ID	CONTRIBUTION
1	*MD. TANVIR ISLAM SARKER TAMIM*	**23-50786-1**	Advance PL/SQL with Exception Handling (Stored Functions, Procedures, Packages, Triggers)
2	*TANVIR MAHATAB ANAN*	**23-50807-1**	Project Updates, Implicit Locking, Explicit Locking, Relational Algebra, DB connection
3	*JANNATUN NESA JERIN*	**23-50798-1**	Interface design, Explicit Locking, Advanced Pl/Sql
4	*SHAHNEWAZ AHMED*	**22-48979-3**	Documentation

CONTENTS

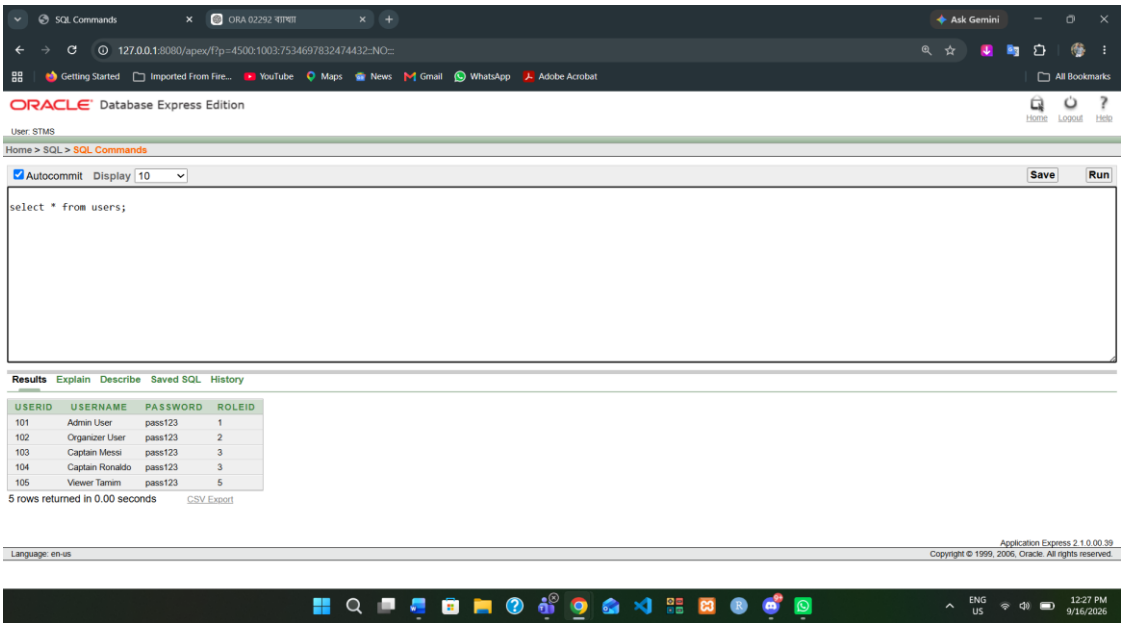
SL	Title	Page Number
1	**Table of Content**	**1**
2	PROJECT UPDATES	**2-3**
3	IMPLICIT LOCKING	**4**
4	EXPLICIT LOCKING	**5**
5	ADVANCE PL/SQL	**6-24**
6	RELATIONAL ALGEBRA	**25**
7	USER INTERFACE	**26-27**
8	DATABASE CONNECTION	**28-30**

PROJECT UPDATES

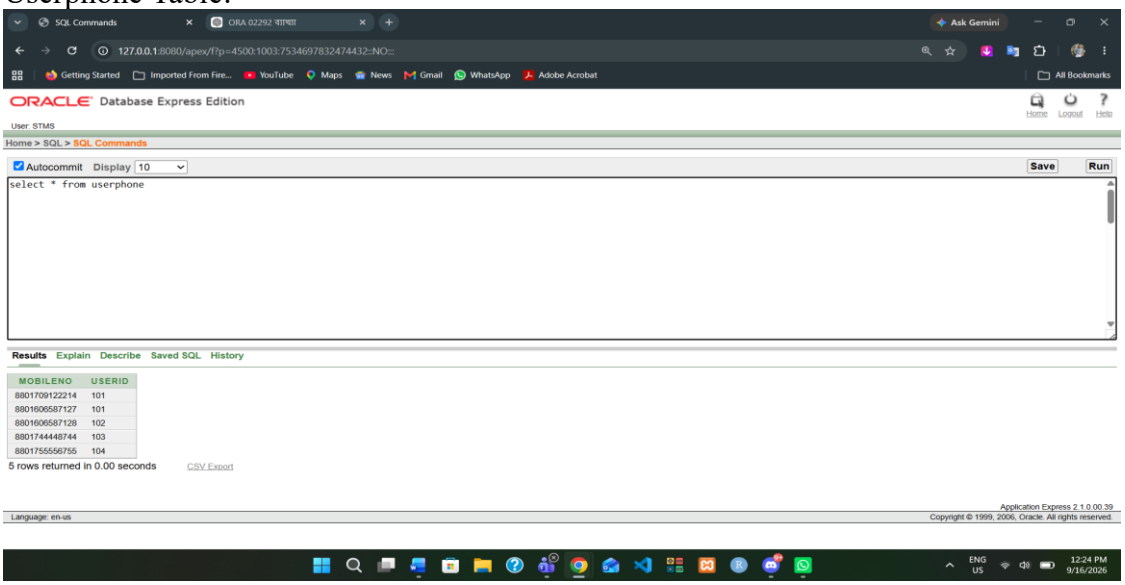
1. **Football Team Data:** The inconsistency found in the football team data was identified and corrected to ensure consistency and accuracy across the database.

   Corrected Tables Screenshots:

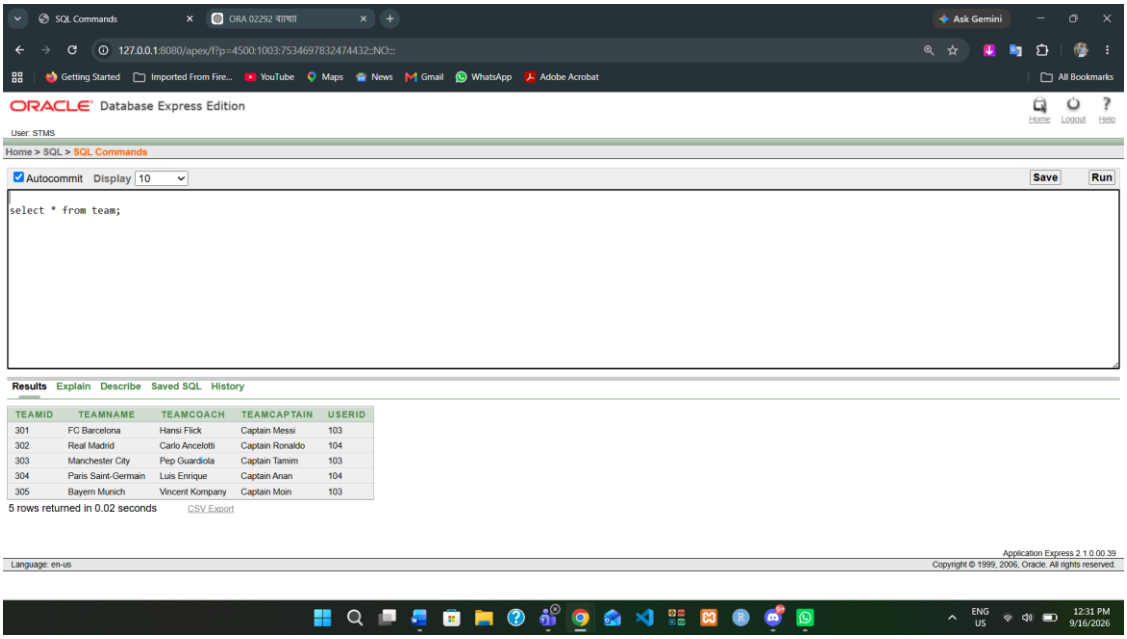
   Users Table:



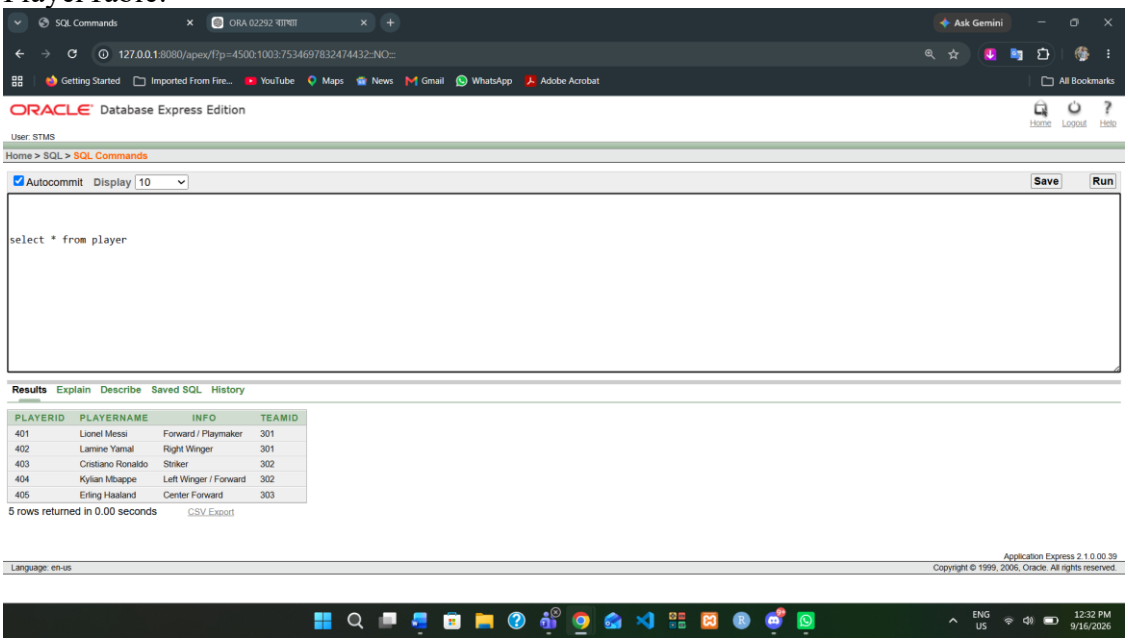
   Userphone Table:



Team Table:



PlayerTable:

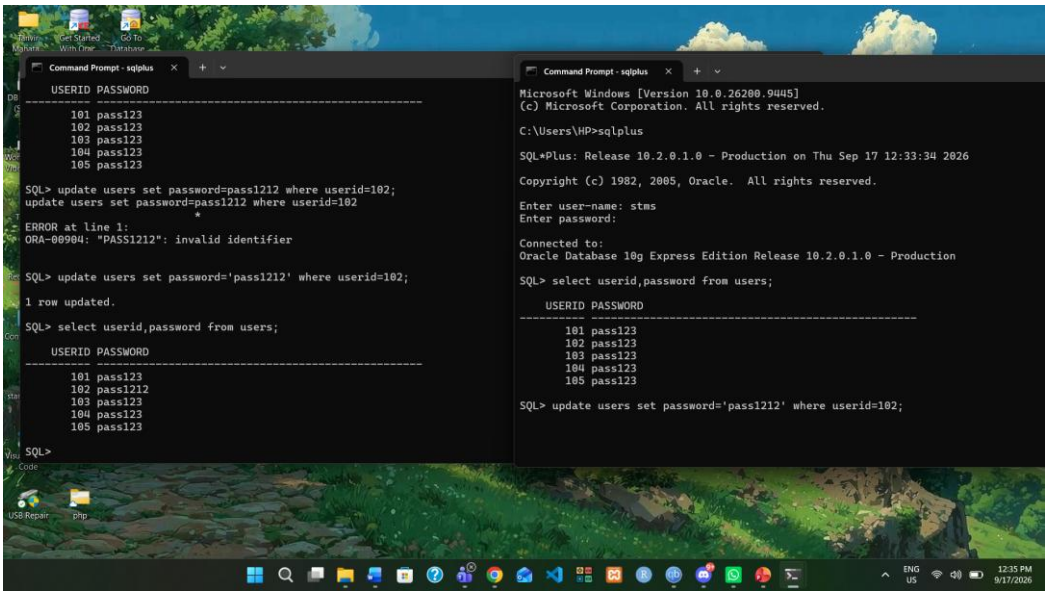


Note: *The representation of roles in the ER diagram has been thoroughly reviewed and verified, with particular attention given to the appropriate normalization of the Role entity. Relevant and valid references supporting the normalization approach have been identified and shared through the Teams message. Furthermore, all necessary modifications and updates have been*

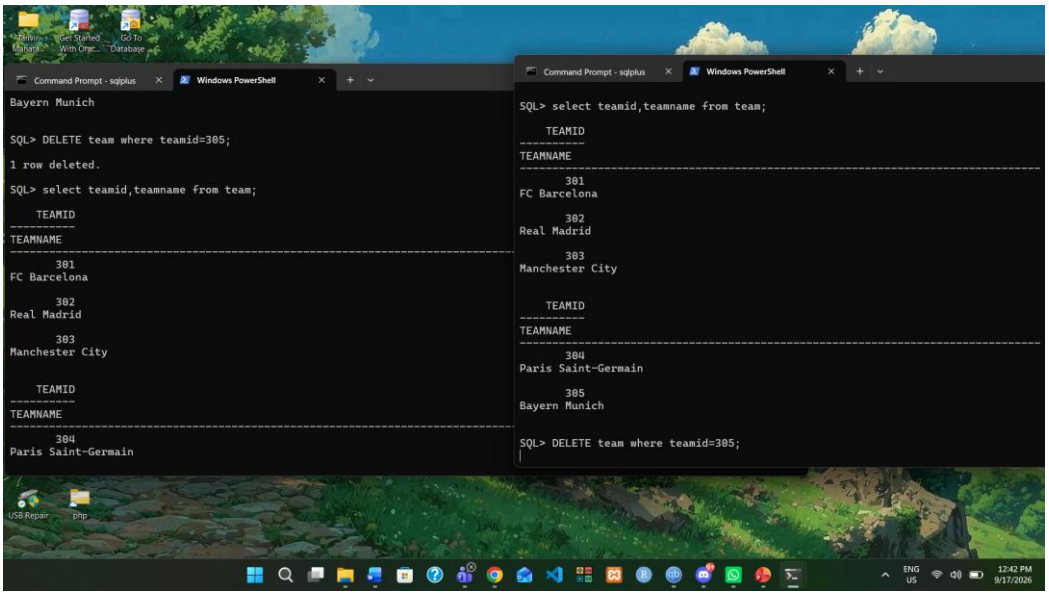
QUERY WRITTING

IMPLICIT LOCKING

1.SQL>UPDATE users SET password=’pass1212’ WHERE userid=102;

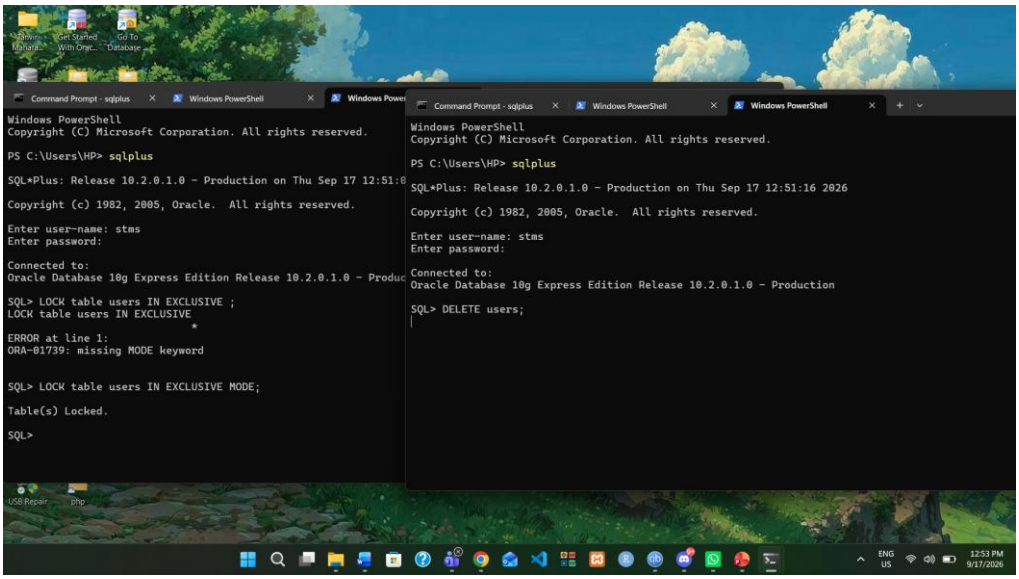


2. SQL>DELETE team where teamid=305;

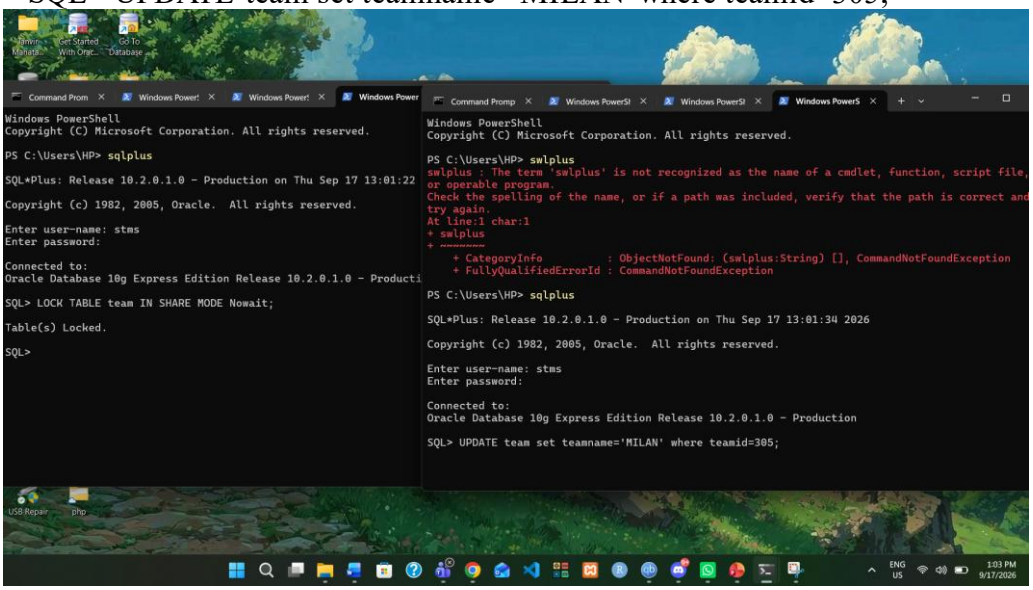


EXPLICIT LOCKING

1. SQL> LOCK table users IN EXCLUSIVE MODE;
   SQL> DELETE users;



2.SQL> LOCK TABLE team IN SHARE MODE Nowait ;
   SQL> UPDATE team set teamname='MILAN' where teamid=305;



ADVANCE PL/SQL

Stored Functions

Question 1:

Write a PL/SQL Stored Function to return the total number of players registered in a specific team with exception handling.

Code:

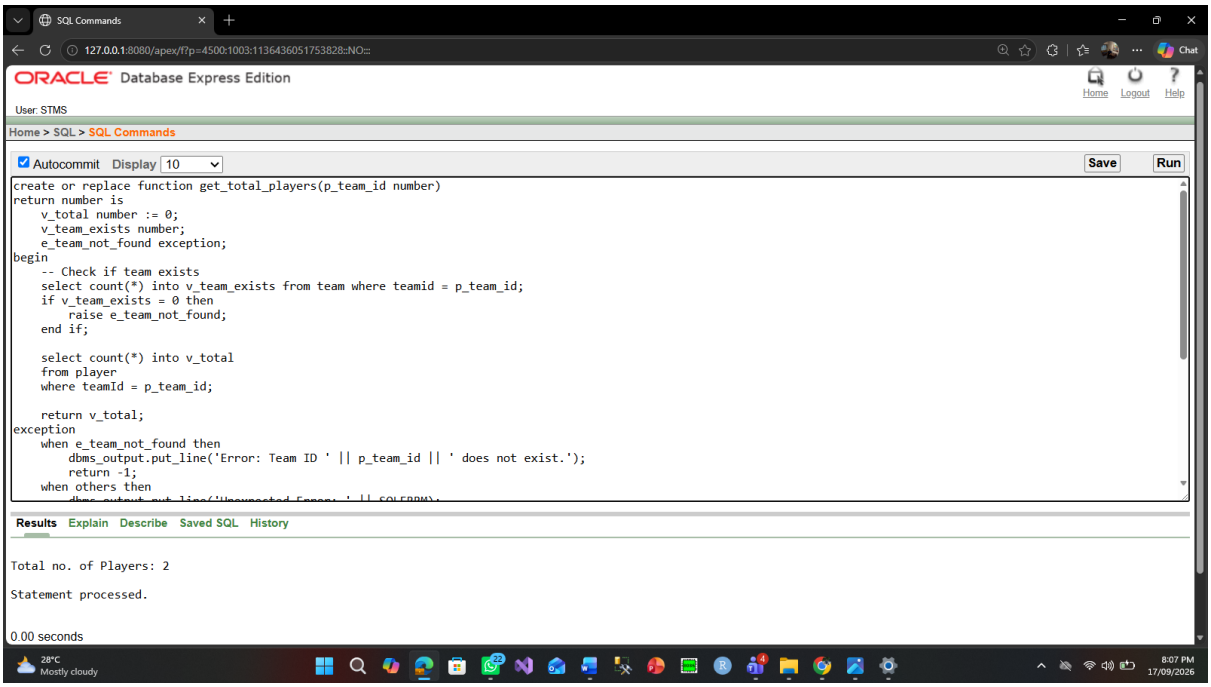
```
create or replace function get_total_players(p_team_id number)
return number is
   v_total number := 0;
   v_team_exists number;
   e_team_not_found exception;
begin
   -- Check if team exists
   select count(*) into v_team_exists from team where teamid = p_team_id;
   if v_team_exists = 0 then
      raise e_team_not_found;
   end if;

   select count(*) into v_total
   from player
   where teamId = p_team_id;

   return v_total;
exception
   when e_team_not_found then
      dbms_output.put_line('Error: Team ID ' || p_team_id || ' does not exist.');
      return -1;
   when others then
      dbms_output.put_line('Unexpected Error: ' || SQLERRM);
      return -1;
end;
/

-- Execution Block:
declare
   c number;
begin
   c := get_total_players(301);
```

```
    dbms_output.put_line('Total no. of Players: ' || c);
exception
    when others then
        dbms_output.put_line('Execution block failed: ' || SQLERRM);
end;
                                                                    /
```



Question 2:

Write a PL/SQL Stored Function to calculate the total duration (in days) of a specific tournament with exception handling.

Code:

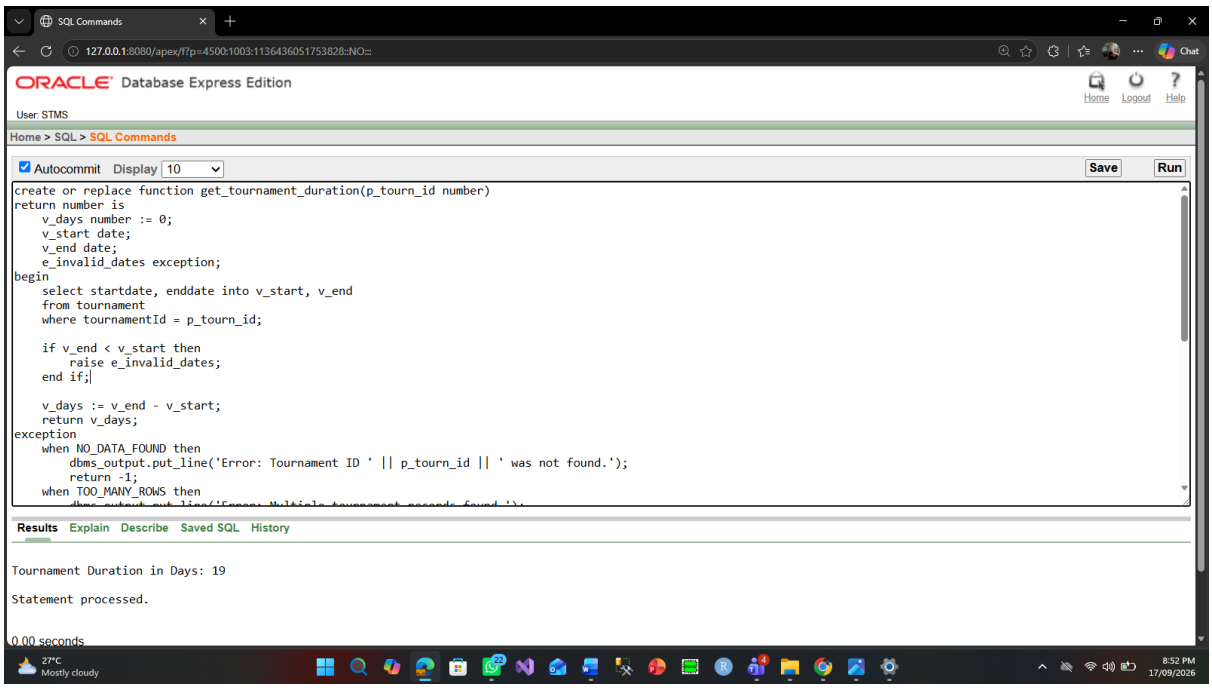
```
create or replace function get_tournament_duration(p_tourn_id number)
return number is
    v_days number := 0;
    v_start date;
    v_end date;
    e_invalid_dates exception;
begin
    select startdate, enddate into v_start, v_end
    from tournament
    where tournamentId = p_tourn_id;

    if v_end < v_start then
        raise e_invalid_dates;
    end if;

    v_days := v_end - v_start;
    return v_days;
```

```
exception
   when NO_DATA_FOUND then
      dbms_output.put_line('Error: Tournament ID ' || p_tourn_id || ' was not found.');
      return -1;
   when TOO_MANY_ROWS then
      dbms_output.put_line('Error: Multiple tournament records found.');
      return -1;
   when e_invalid_dates then
      dbms_output.put_line('Error: End date is earlier than start date.');
      return -1;
   when others then
      dbms_output.put_line('Error Code: ' || SQLCODE || ' - ' || SQLERRM);
      return -1;
end;
/

-- Execution Block:
declare
   c number;
begin
   c := get_tournament_duration(201);
   dbms_output.put_line('Tournament Duration in Days: ' || c);
exception
   when others then
      dbms_output.put_line('Failed: ' || SQLERRM);
end;
/
```



Question 3:
Write a PL/SQL Stored Procedure to update the status of a match based on the provided match ID with exception handling.
Code:

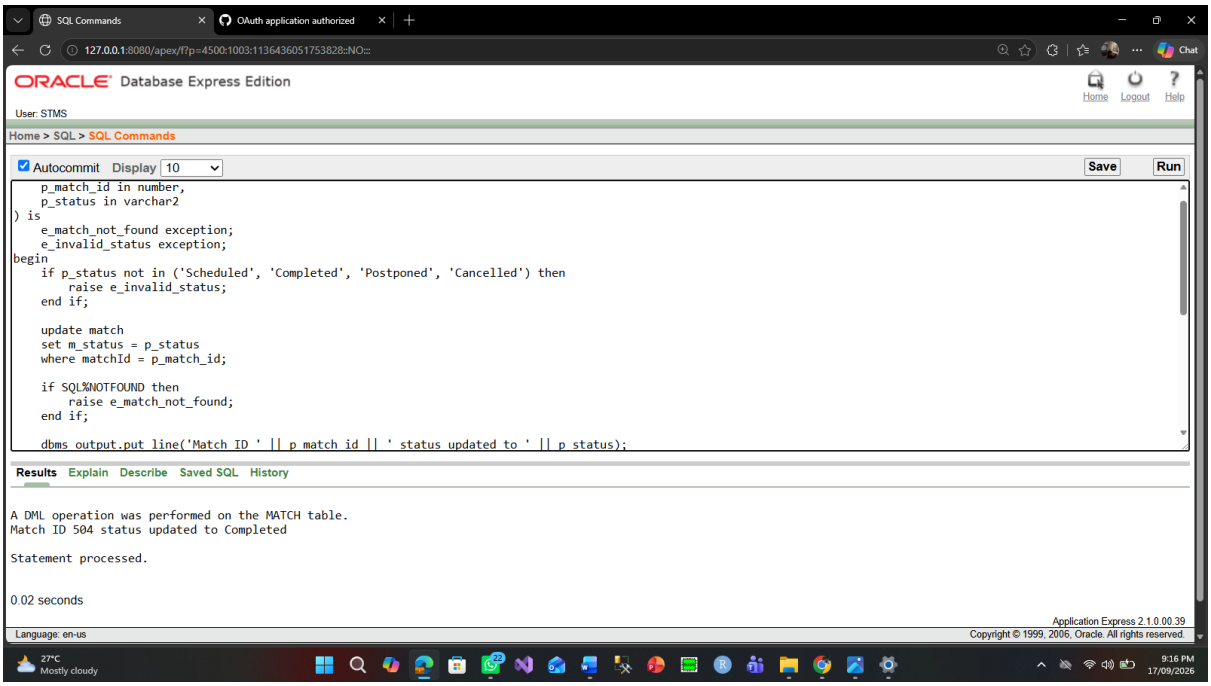
```
create or replace procedure update_match_status(
   p_match_id in number,
   p_status in varchar2
) is
   e_match_not_found exception;
   e_invalid_status exception;
begin
   if p_status not in ('Scheduled', 'Completed', 'Postponed', 'Cancelled') then
      raise e_invalid_status;
   end if;

   update match
   set m_status = p_status
   where matchId = p_match_id;

   if SQL%NOTFOUND then
      raise e_match_not_found;
   end if;

   dbms_output.put_line('Match ID ' || p_match_id || ' status updated to ' || p_status);
exception
   when e_invalid_status then
      dbms_output.put_line('Validation Error: Status must be Scheduled, Completed, Postponed, or
Cancelled.');
   when e_match_not_found then
      dbms_output.put_line('Error: Match ID ' || p_match_id || ' does not exist in Match table.');
   when others then
      dbms_output.put_line('Error (' || SQLCODE || '): ' || SQLERRM);
end;
/

-- Execution Block:
begin
   update_match_status(504, 'Completed');
exception
   when others then
      dbms_output.put_line('Execution failed: ' || SQLERRM);
end;
```



Question 4:

Write a PL/SQL Stored Procedure to register a team for a tournament by handling unique and foreign key violations.

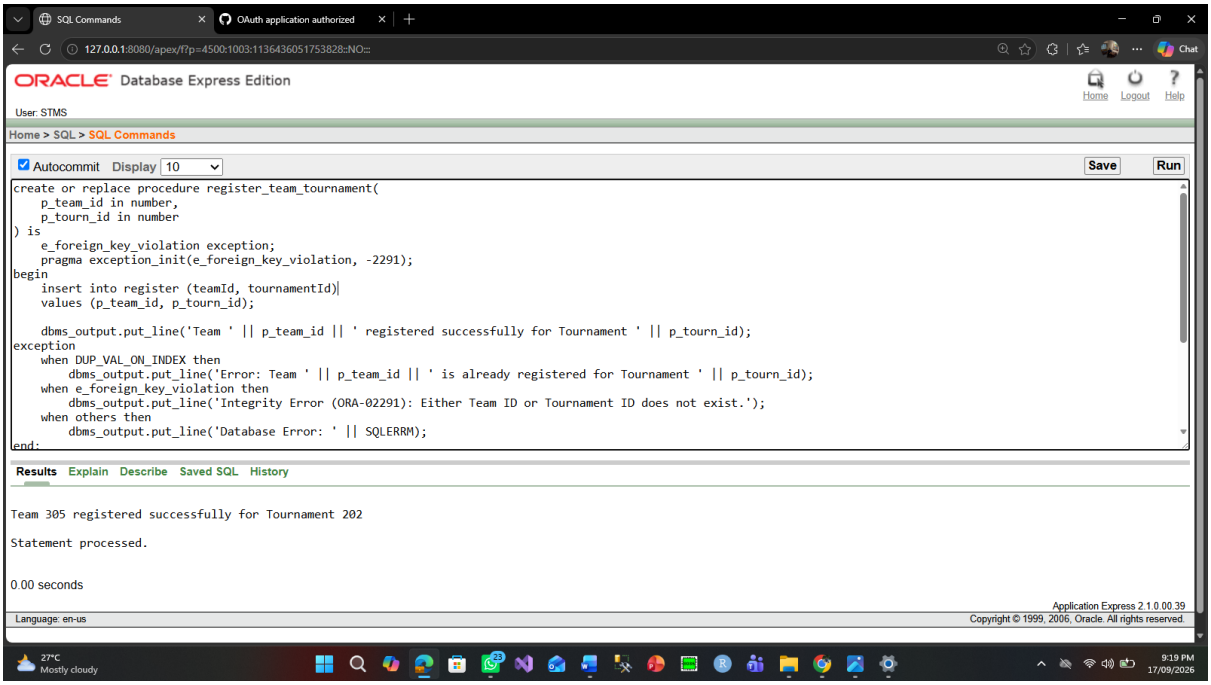
Code:

```
create or replace procedure register_team_tournament(
    p_team_id in number,
    p_tourn_id in number
) is
    e_foreign_key_violation exception;
    pragma exception_init(e_foreign_key_violation, -2291);
begin
    insert into register (teamId, tournamentId)
    values (p_team_id, p_tourn_id);

    dbms_output.put_line('Team ' || p_team_id || ' registered successfully for Tournament ' || p_tourn_id);
exception
    when DUP_VAL_ON_INDEX  then
        dbms_output.put_line('Error: Team ' || p_team_id || ' is already registered for Tournament ' || p_tourn_id);
    when e_foreign_key_violation then
        dbms_output.put_line('Integrity Error (ORA-02291): Either Team ID or Tournament ID does not exist.');
    when others then
        dbms_output.put_line('Database Error: ' || SQLERRM);
end;
/

-- Execution Block:
begin
```

```
    register_team_tournament(305, 202);
exception
    when others then
        dbms_output.put_line('Procedure call failed: ' || SQLERRM);
end;
/
```



Question 5:

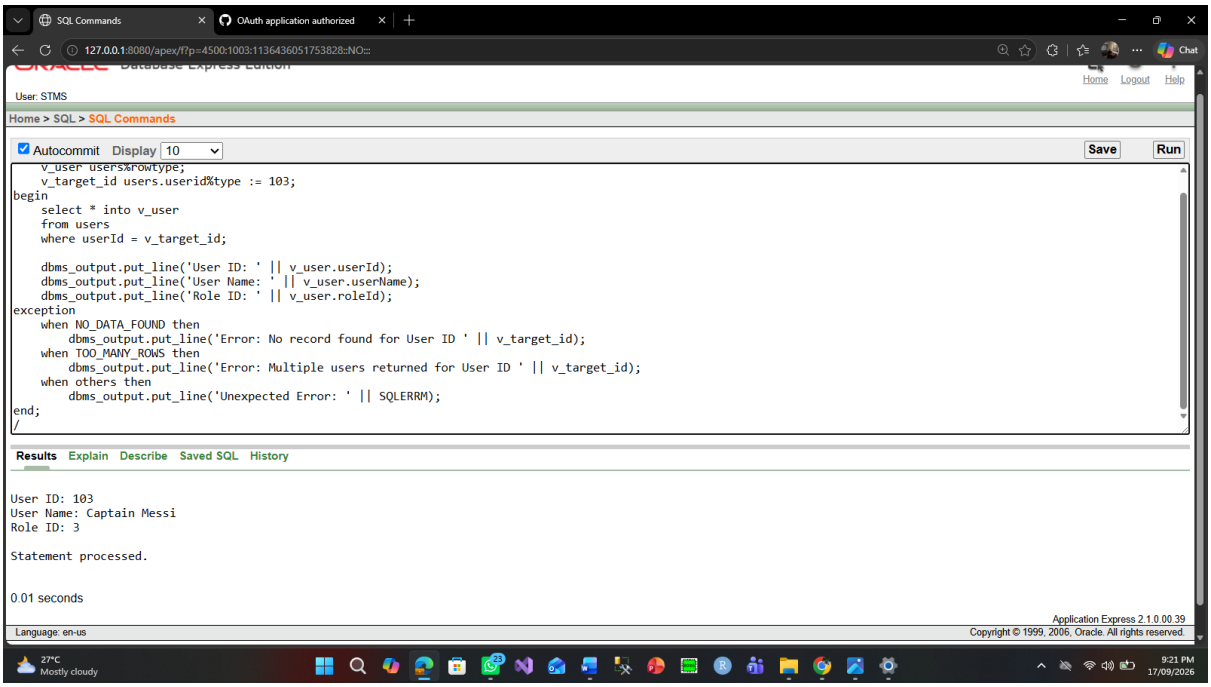
Write a PL/SQL program using a table-based record (%rowtype) to retrieve and display full details of a specific user with exception handling.

Code:

```
declare
    v_user users%rowtype;
    v_target_id users.userid%type := 103;
begin
    select * into v_user
    from users
    where userId = v_target_id;

    dbms_output.put_line('User ID: ' || v_user.userId);
    dbms_output.put_line('User Name: ' || v_user.userName);
    dbms_output.put_line('Role ID: ' || v_user.roleId);
exception
    when NO_DATA_FOUND then
        dbms_output.put_line('Error: No record found for User ID ' || v_target_id);
    when TOO_MANY_ROWS then
        dbms_output.put_line('Error: Multiple users returned for User ID ' || v_target_id);
    when others then
        dbms_output.put_line('Unexpected Error: ' || SQLERRM);
end;
```

/



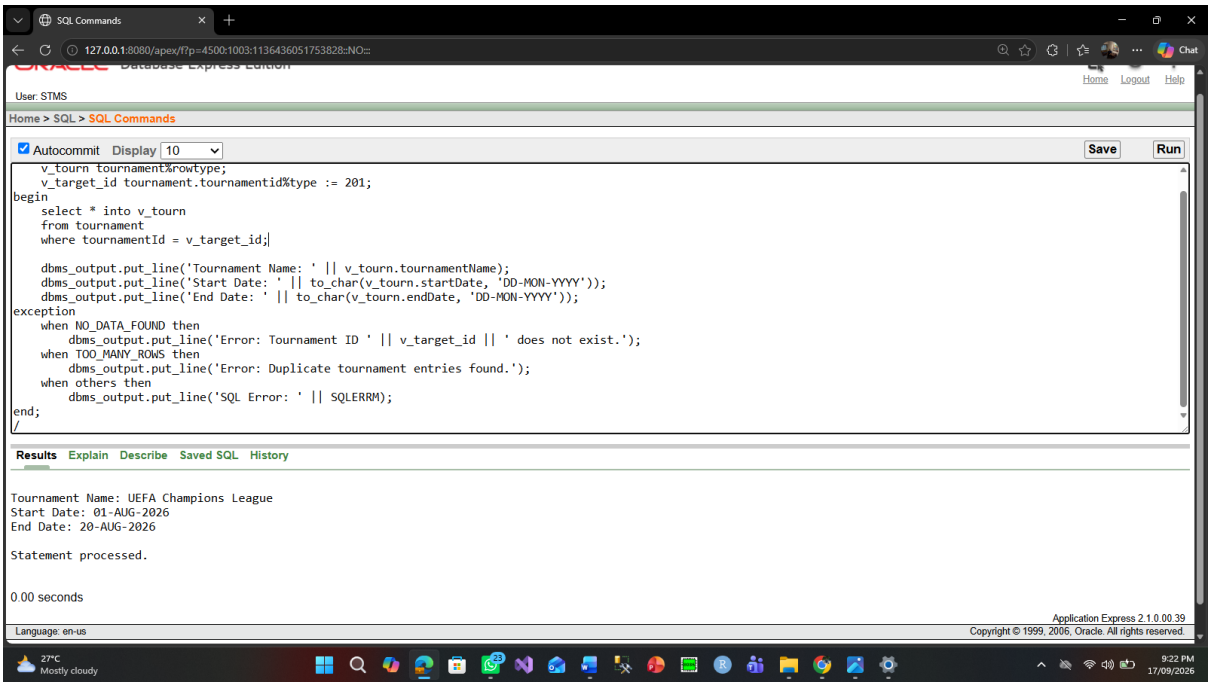
Question 6:

Write a PL/SQL program using a table-based record (%rowtype) to fetch and display tournament details with exception handling.

Code:

```
declare
   v_tourn tournament%rowtype;
   v_target_id tournament.tournamentid%type := 201;
begin
   select * into v_tourn
   from tournament
   where tournamentId = v_target_id;

   dbms_output.put_line('Tournament Name: ' || v_tourn.tournamentName);
   dbms_output.put_line('Start Date: ' || to_char(v_tourn.startDate, 'DD-MON-YYYY'));
   dbms_output.put_line('End Date: ' || to_char(v_tourn.endDate, 'DD-MON-YYYY'));
exception
   when NO_DATA_FOUND then
      dbms_output.put_line('Error: Tournament ID ' || v_target_id || ' does not exist.');
   when TOO_MANY_ROWS then
      dbms_output.put_line('Error: Duplicate tournament entries found.');
   when others then
      dbms_output.put_line('SQL Error: ' || SQLERRM);
end;
/
```



Question 7:

Write a PL/SQL block using an Explicit Cursor to iterate through all matches and handle cursor exceptions.

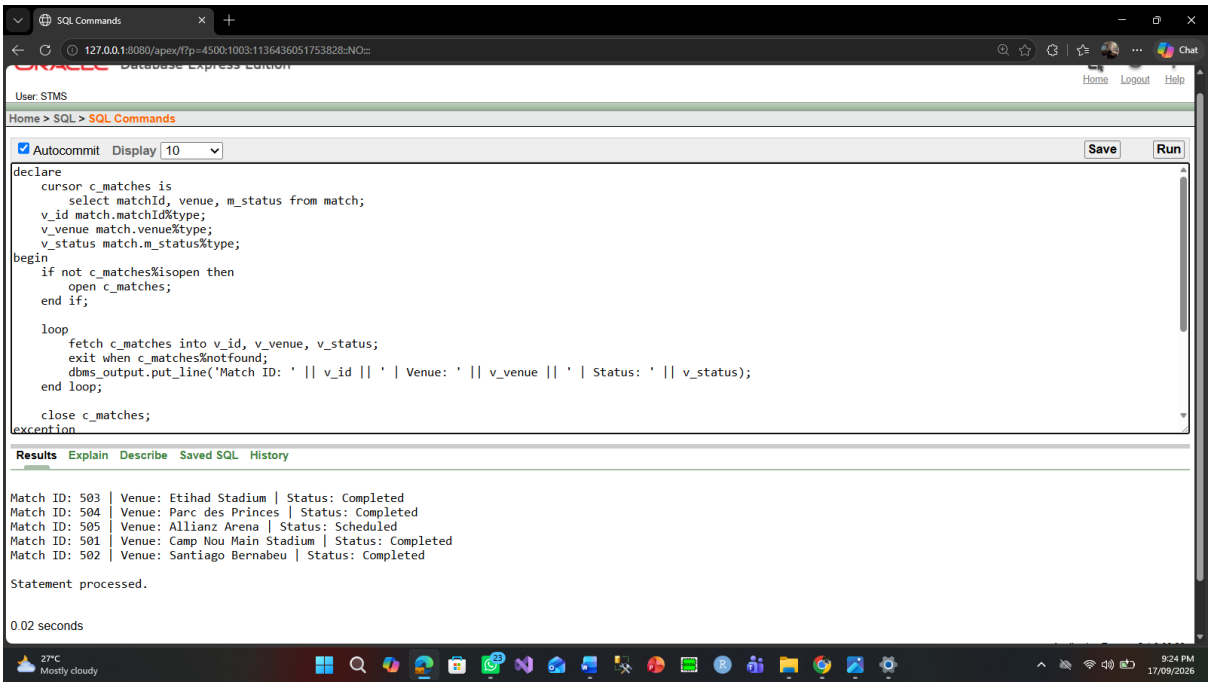
Code:

```
declare
   cursor c_matches is
      select matchId, venue, m_status from match;
   v_id match.matchId%type;
   v_venue match.venue%type;
   v_status match.m_status%type;
begin
   if not c_matches%isopen then
      open c_matches;
   end if;

   loop
      fetch c_matches into v_id, v_venue, v_status;
      exit when c_matches%notfound;
      dbms_output.put_line('Match ID: ' || v_id || ' | Venue: ' || v_venue || ' | Status: ' || v_status);
   end loop;

   close c_matches;
exception
   when INVALID_CURSOR then
      dbms_output.put_line('Cursor Error: An operation was attempted on an invalid/closed cursor.');
   when others then
      if c_matches%isopen then
         close c_matches;
      end if;
```

```
    dbms_output.put_line('Cursor Trapped Error: ' || SQLERRM);
end;
/
```



Question 8:

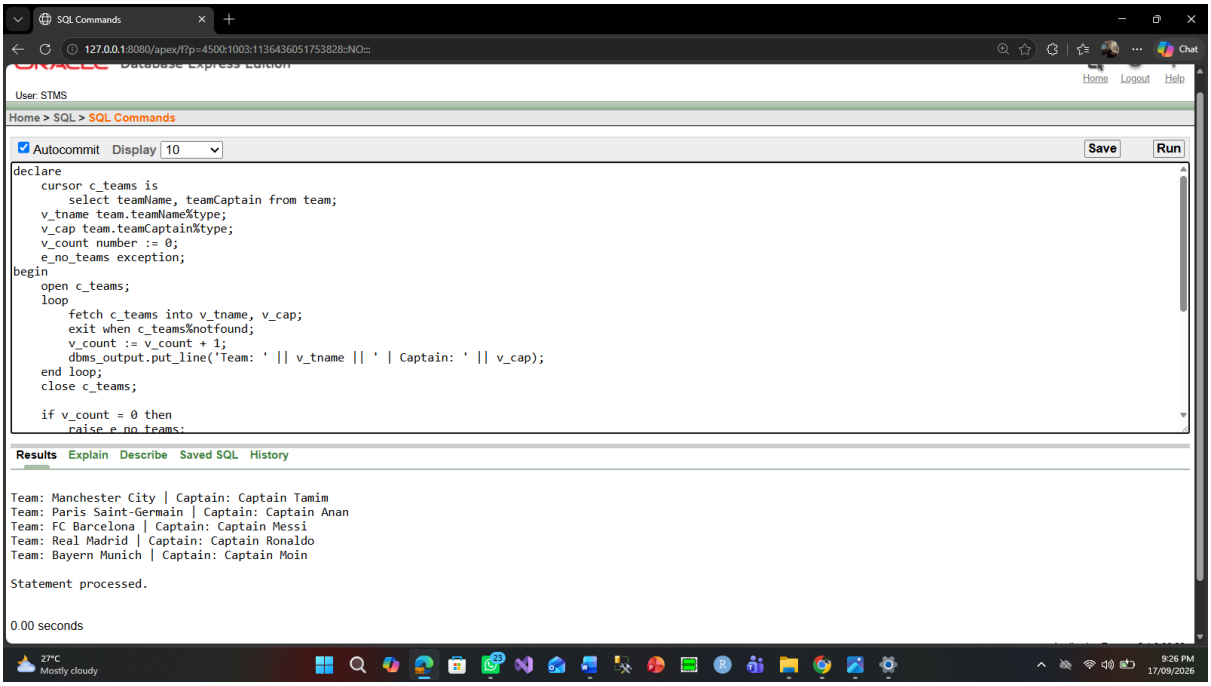
Write a PL/SQL block using an Explicit Cursor to display teams and their captains, with exception handling.

Code:

```
declare
   cursor c_teams is
      select teamName, teamCaptain from team;
   v_tname team.teamName%type;
   v_cap team.teamCaptain%type;
   v_count number := 0;
   e_no_teams exception;
begin
   open c_teams;
   loop
      fetch c_teams into v_tname, v_cap;
      exit when c_teams%notfound;
      v_count := v_count + 1;
      dbms_output.put_line('Team: ' || v_tname || ' | Captain: ' || v_cap);
   end loop;
   close c_teams;

   if v_count = 0 then
      raise e_no_teams;
   end if;
exception
   when e_no_teams then
```

```
        dbms_output.put_line('Notice: No team records found in the database.');
    when INVALID_CURSOR then
        dbms_output.put_line('Error: Cursor handling error encountered.');
    when others then
        if c_teams%isopen then
            close c_teams;
        end if;
        dbms_output.put_line('Unexpected Error: ' || SQLERRM);
end;
/
```



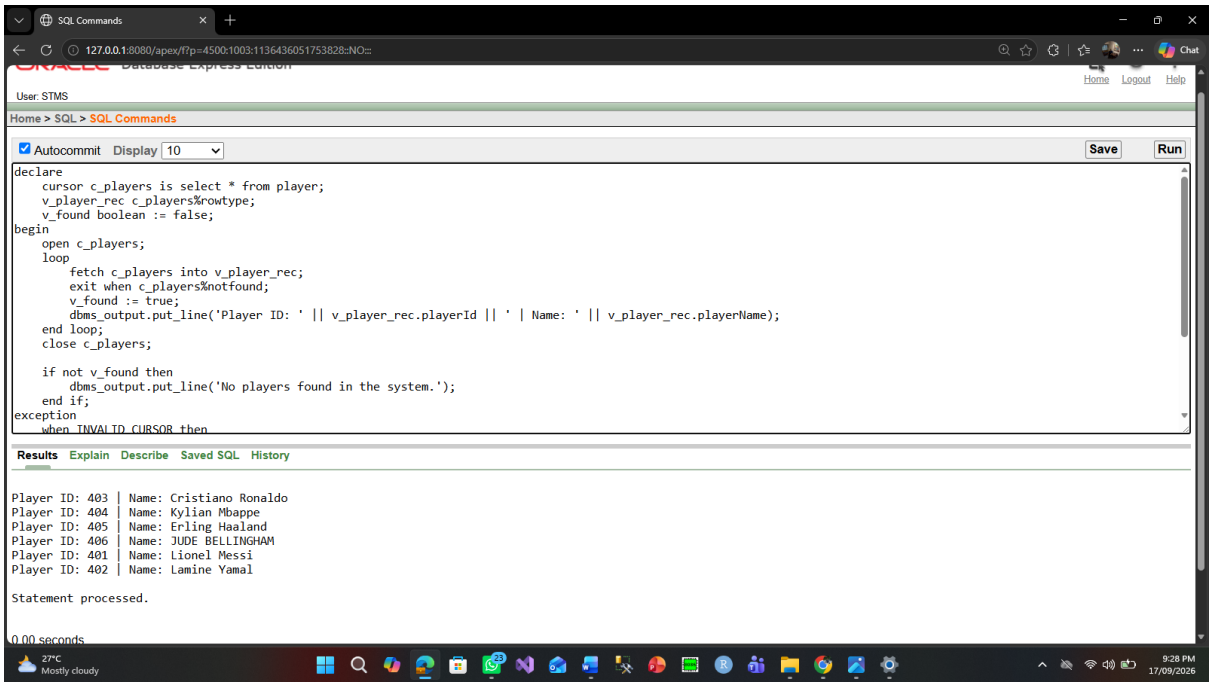
Question 9:

Write a PL/SQL program combining an Explicit Cursor with a cursor-based record (c_players%rowtype) with exception handling.

Code:

```
declare
    cursor c_players is select * from player;
    v_player_rec c_players%rowtype;
    v_found boolean := false;
begin
    open c_players;
    loop
        fetch c_players into v_player_rec;
        exit when c_players%notfound;
        v_found := true;
        dbms_output.put_line('Player ID: ' || v_player_rec.playerId || ' | Name: ' || v_player_rec.playerName);
    end loop;
    close c_players;
```

```
    if not v_found then
        dbms_output.put_line('No players found in the system.');
    end if;
exception
    when INVALID_CURSOR then
        dbms_output.put_line('Error: Invalid cursor state.');
    when others then
        if c_players%isopen then
            close c_players;
        end if;
        dbms_output.put_line('Execution failed: ' || SQLCODE || ' - ' || SQLERRM);
end;
/
```



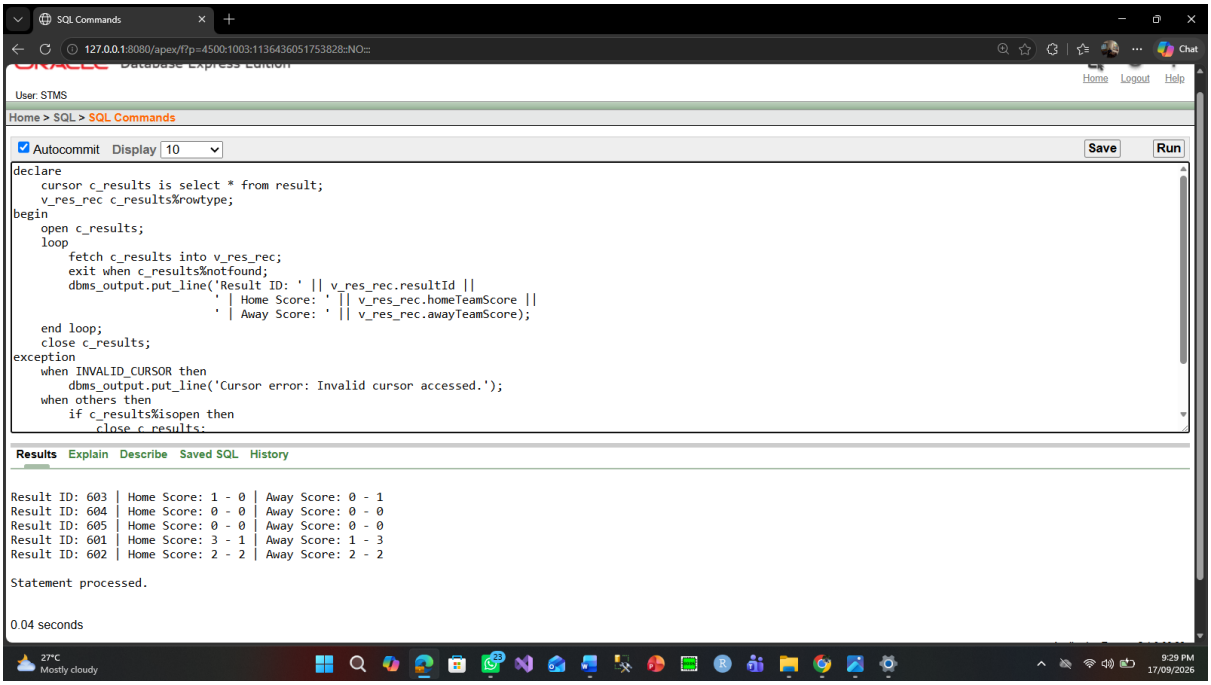
Question 10:

Write a PL/SQL program using a cursor-based record (c_results%rowtype) to display all match results and scores with exception handling.

Code:

```
declare
    cursor c_results is select * from result;
    v_res_rec c_results%rowtype;
begin
    open c_results;
    loop
        fetch c_results into v_res_rec;
        exit when c_results%notfound;
        dbms_output.put_line('Result ID: ' || v_res_rec.resultId ||
```

```
                        ' | Home Score: ' || v_res_rec.homeTeamScore ||
                        ' | Away Score: ' || v_res_rec.awayTeamScore);
    end loop;
    close c_results;
exception
    when INVALID_CURSOR then
        dbms_output.put_line('Cursor error: Invalid cursor accessed.');
    when others then
        if c_results%isopen then
            close c_results;
        end if;
        dbms_output.put_line('Error encountered: ' || SQLERRM);
end;
/
```



Question 11:

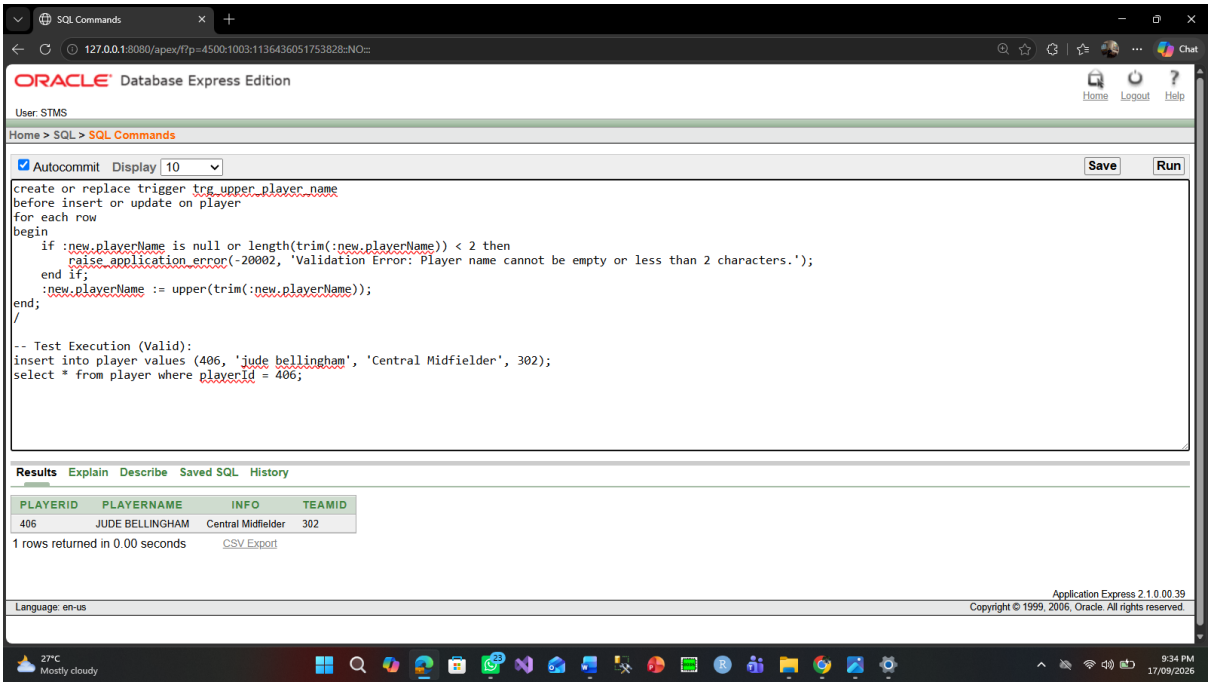
Create a Row-Level Trigger that formats player name to UPPERCASE and raises an application error if player name is null or too short.

Code:

```
create or replace trigger trg_upper_player_name
before insert or update on player
for each row
begin
    if :new.playerName is null or length(trim(:new.playerName)) < 2 then
        raise_application_error(-20002, 'Validation Error: Player name cannot be empty or less than 2
characters.');
    end if;
    :new.playerName := upper(trim(:new.playerName));
end;
```

/

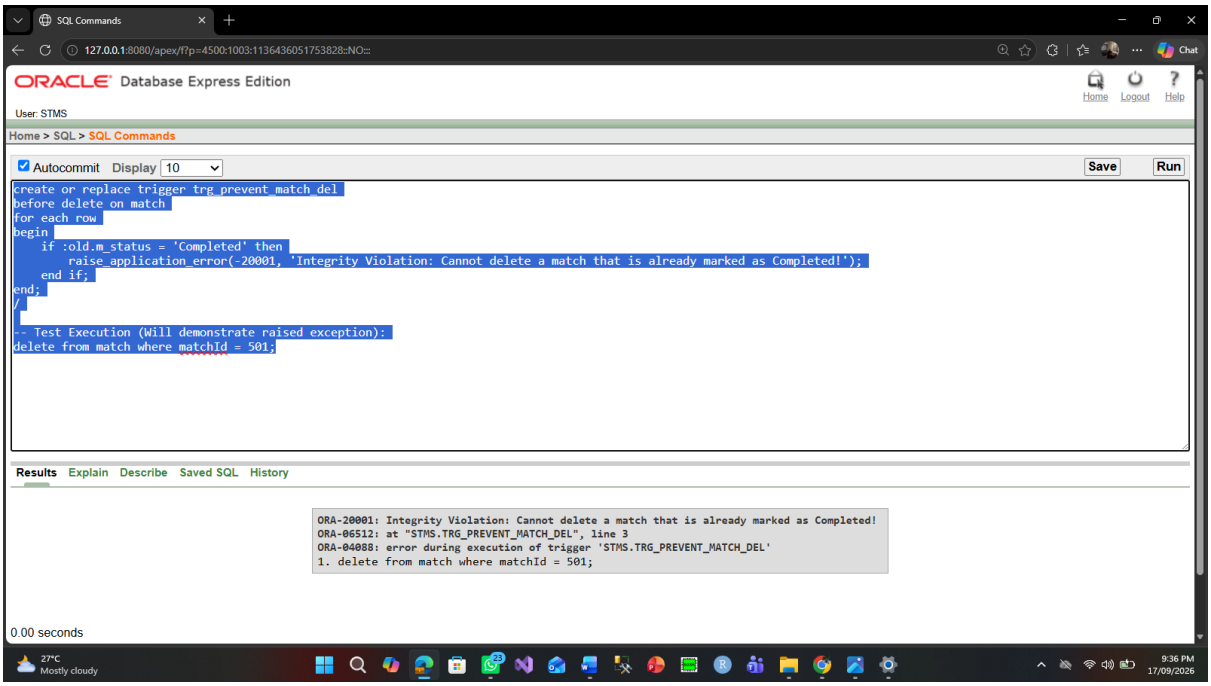
-- Test Execution (Valid):
insert into player values (406, 'jude bellingham', 'Central Midfielder', 302);
select * from player where playerId = 406;



Question 12:
Create a Row-Level Trigger to prevent deletion of completed matches using RAISE_APPLICATION_ERROR.
Code:

```
create or replace trigger trg_prevent_match_del
before delete on match
for each row
begin
   if :old.m_status = 'Completed' then
      raise_application_error(-20001, 'Integrity Violation: Cannot delete a match that is already marked as Completed!');
   end if;
end;
/

-- Test Execution (Will demonstrate raised exception):
delete from match where matchId = 501;
```



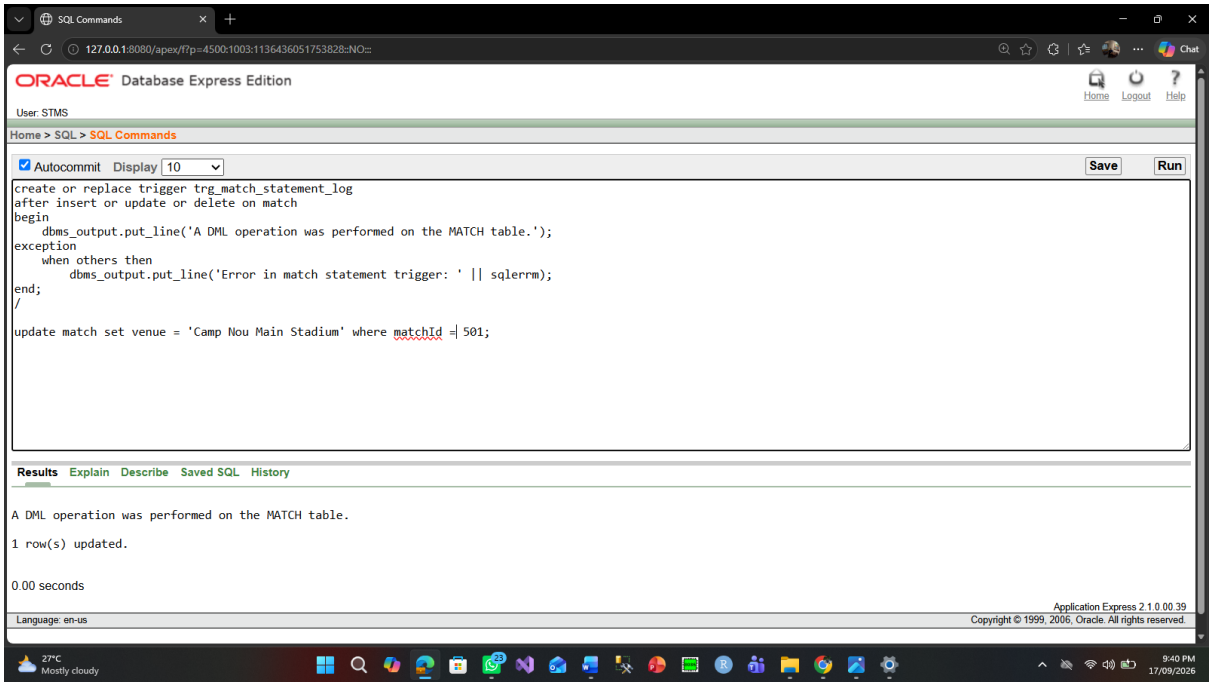
Question 13:

Create a Statement-Level Trigger that prints a notification whenever an INSERT, UPDATE, or DELETE operation occurs on the Match table with exception handling.

Code:

```
create or replace trigger trg_match_statement_log
after insert or update or delete on match
begin
   dbms_output.put_line('A DML operation was performed on the MATCH table.');
exception
   when others then
      dbms_output.put_line('Error in match statement trigger: ' || sqlerrm);
end;
/

update match set venue = 'Camp Nou Main Stadium' where matchId = 501;
```



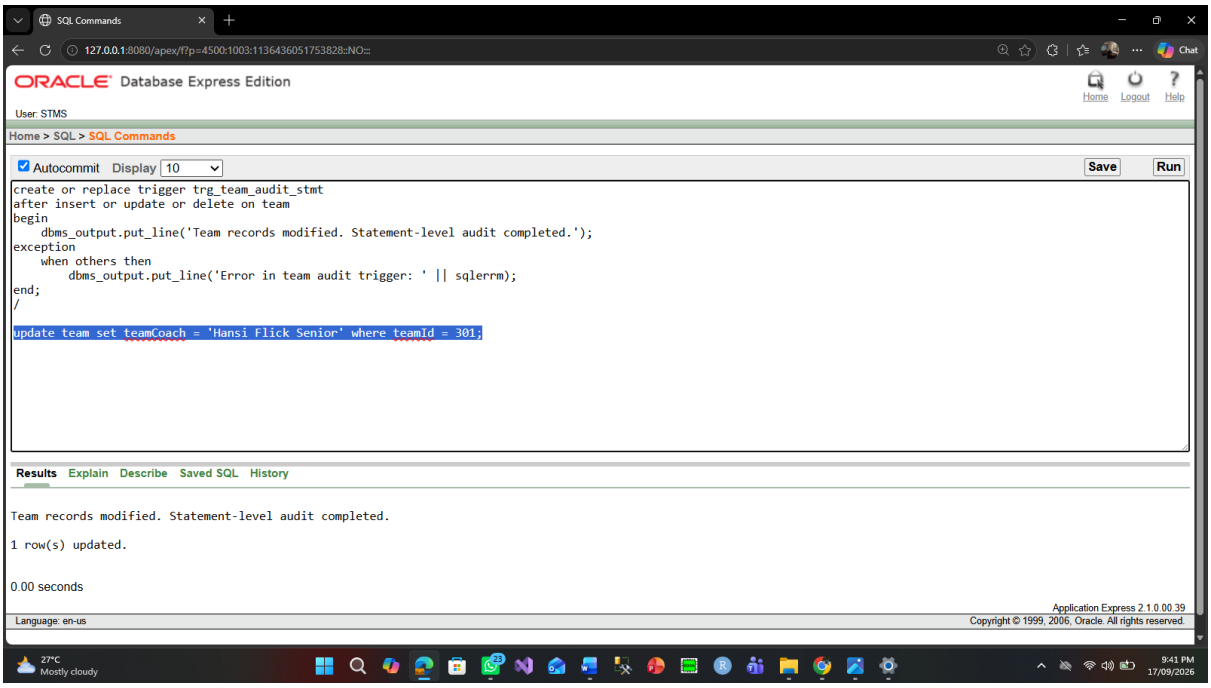
Question 14:

Create a Statement-Level Trigger that logs an audit message whenever modifications are made to the Team table with exception handling.

Code:

```
create or replace trigger trg_team_audit_stmt
after insert or update or delete on team
begin
   dbms_output.put_line('Team records modified. Statement-level audit completed.');
exception
   when others then
      dbms_output.put_line('Error in team audit trigger: ' || sqlerrm);
end;
/


update team set teamCoach = 'Hansi Flick Senior' where teamId = 301;
```



Question 15:

Create a Package (pkg_tournament) consisting of procedures and functions with comprehensive exception handling.

Code:

```
create or replace package pkg_tournament as
   procedure add_tournament(p_id number, p_name varchar2, p_start date, p_end date);
   function get_name(p_id number) return varchar2;
end pkg_tournament;
/

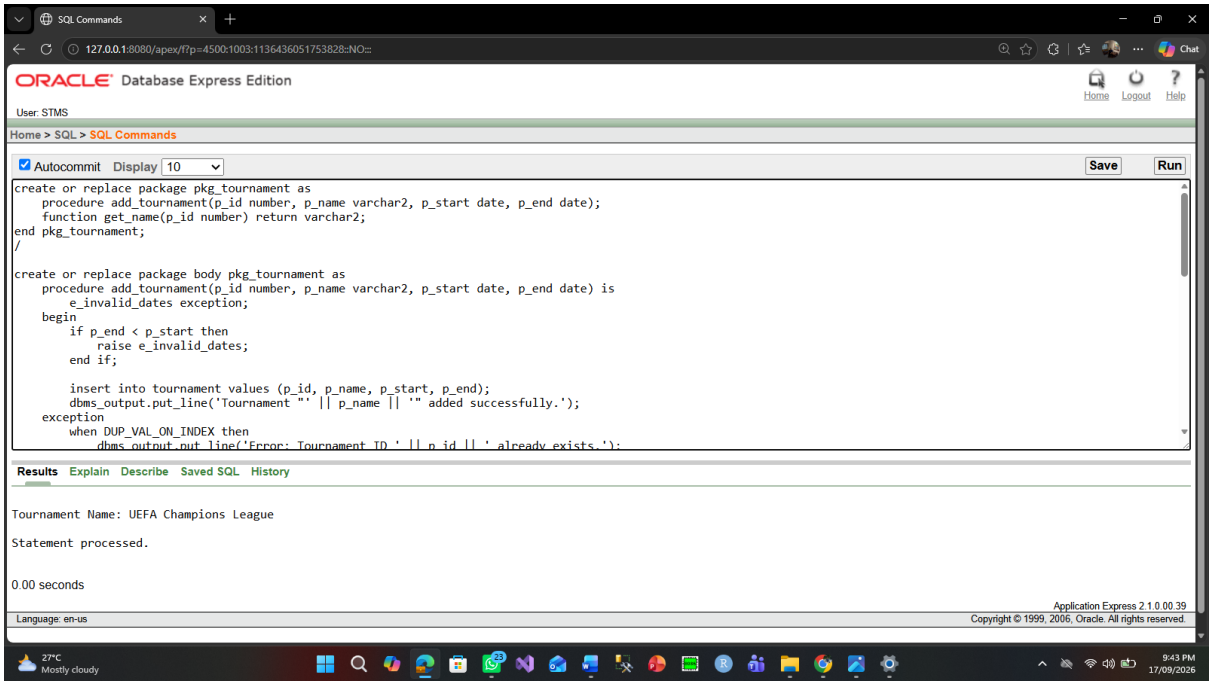
create or replace package body pkg_tournament as
   procedure add_tournament(p_id number, p_name varchar2, p_start date, p_end date) is
      e_invalid_dates exception;
   begin
      if p_end < p_start then
         raise e_invalid_dates;
      end if;

      insert into tournament values (p_id, p_name, p_start, p_end);
      dbms_output.put_line('Tournament ''' || p_name || ''' added successfully.');
   exception
      when DUP_VAL_ON_INDEX  then
         dbms_output.put_line('Error: Tournament ID ' || p_id || ' already exists.');
      when e_invalid_dates then
         dbms_output.put_line('Error: Tournament End Date cannot precede Start Date.');
      when others then
         dbms_output.put_line('Package Procedure Error: ' || SQLERRM);
```

```
    end add_tournament;

    function get_name(p_id number) return varchar2 is
        v_name varchar2(100);
    begin
        select tournamentName into v_name from tournament where tournamentId = p_id;
        return v_name;
    exception
        when NO_DATA_FOUND  then
            return 'Tournament Not Found';
        when TOO_MANY_ROWS then
            return 'Multiple Tournaments Found';
        when others then
            return 'Error: ' || SQLERRM;
    end get_name;
end pkg_tournament;
/

-- Execution Block:
declare
    c varchar2(100);
begin
    c := pkg_tournament.get_name(201);
    dbms_output.put_line('Tournament Name: ' || c);
exception
    when others then
        dbms_output.put_line('Package call failed: ' || SQLERRM);
end;
/
```



Question 16:
Create a Package (pkg_team) containing subprograms for adding players and retrieving team captains with exception handling.
Code:

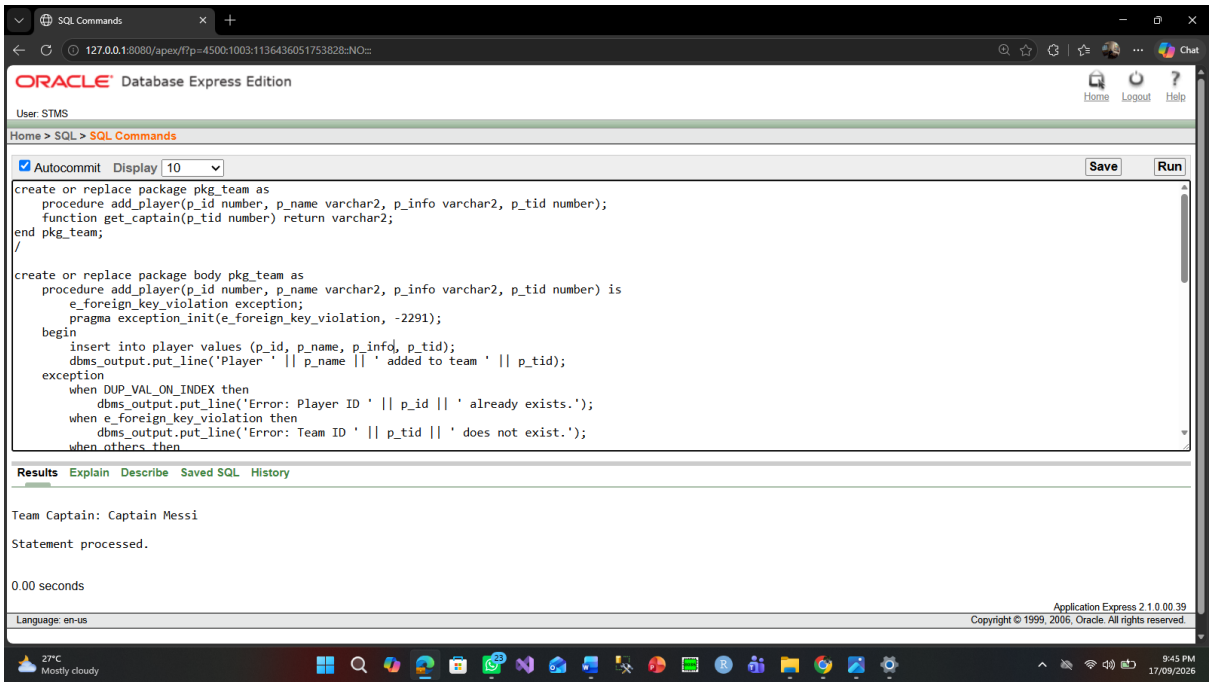
```
create or replace package pkg_team as
   procedure add_player(p_id number, p_name varchar2, p_info varchar2, p_tid number);
   function get_captain(p_tid number) return varchar2;
end pkg_team;
/

create or replace package body pkg_team as
   procedure add_player(p_id number, p_name varchar2, p_info varchar2, p_tid number) is
      e_foreign_key_violation exception;
      pragma exception_init(e_foreign_key_violation, -2291);
   begin
      insert into player values (p_id, p_name, p_info, p_tid);
      dbms_output.put_line('Player ' || p_name || ' added to team ' || p_tid);
   exception
      when DUP_VAL_ON_INDEX then
         dbms_output.put_line('Error: Player ID ' || p_id || ' already exists.');
      when e_foreign_key_violation then
         dbms_output.put_line('Error: Team ID ' || p_tid || ' does not exist.');
      when others then
         dbms_output.put_line('Failed to insert player: ' || SQLERRM);
   end add_player;

   function get_captain(p_tid number) return varchar2 is
      v_cap varchar2(100);
   begin
      select teamCaptain into v_cap from team where teamId = p_tid;
      return v_cap;
   exception
      when NO_DATA_FOUND then
         return 'No Captain Found';
      when TOO_MANY_ROWS then
         return 'Duplicate Captain Records Found';
      when others then
         return 'Error: ' || SQLERRM;
   end get_captain;
end pkg_team;
/

-- Execution Block:
declare
   c varchar2(100);
begin
   c := pkg_team.get_captain(301);
   dbms_output.put_line('Team Captain: ' || c);
exception
```

```
    when others then
        dbms_output.put_line('Error calling pkg_team: ' || SQLERRM);
end;
/
```



RELATIONAL ALGEBRA

1. Find the name of the Teams

   Answer: $\prod_{teamname}(Team)$

2. Find all match in Camp Nou

   Answer: $\sigma_{venue = \text{“Camp Nou”}}(Match)$

3. Find the name of the tournament whose start date is 01-AUG-26.

   Answer: $\prod_{tournamentname}(\sigma_{startdate = \text{“01-AUG-26”}}(Tournament)$

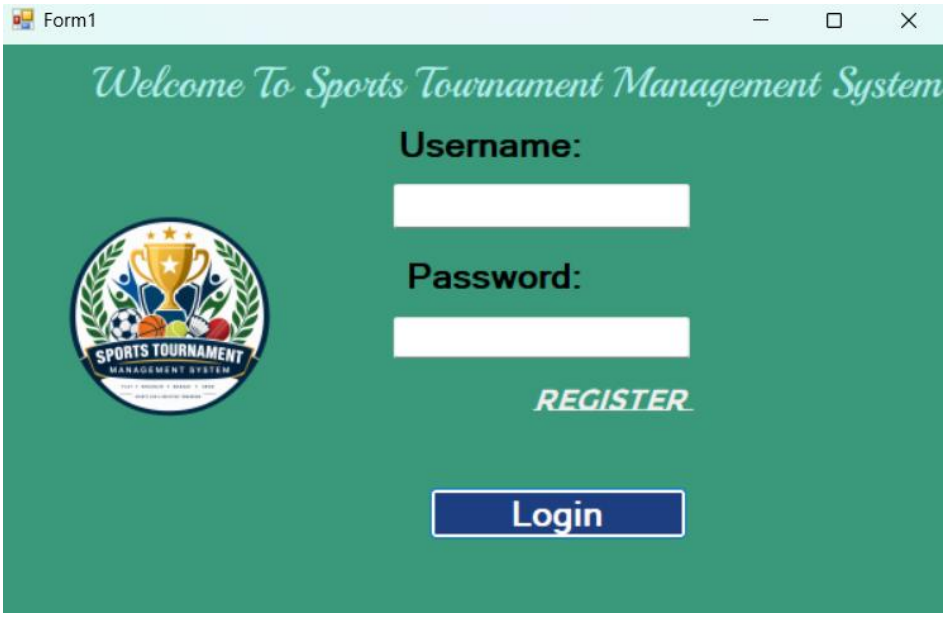
4. Display the Team Name and Team Captain for teams where USERID = 103.

   Answer: $\prod_{teamname,\ teamcaptain}(\sigma_{userid = 103}(\text{TEAM})$

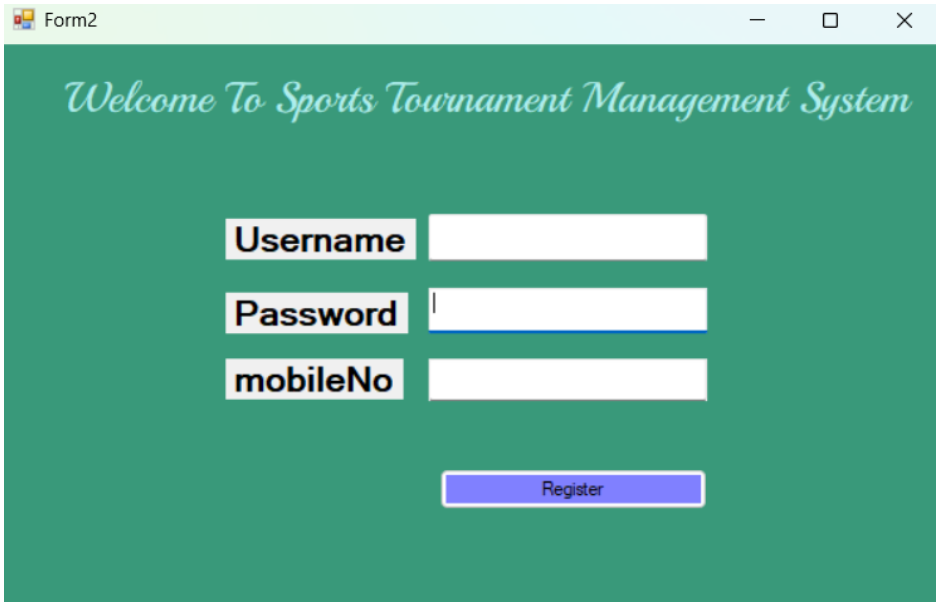
5. Find the TEAMID and TEAMCOACH of teams where USERID = 103 AND TEAMNAME = 'Manchester City'.
   Answer: $\prod_{teamid,\ teamcoach}(\sigma_{userid = 103 \wedge \text{ teamname=” Manchester city”}}(\text{TEAM})$

USER INTERFACE

Login Page



Registration Page:



Dashboard:


Form3
Welcome To Sports Tournament Management System
DASHBOARD
View Team
Match
Scoreboard
SPORTS TOURNAMENT
MANAGEMENT SYSTEM
Developed by Anan,tamim,jerin

DATABASE CONNECTION

The **Sports Tournament Management System** employs **Oracle Database 10g Express Edition** as its backend database and **C# Windows Forms** as the application layer. The communication between the application and the database is established using the **Oracle Managed Data Access** library, which provides the necessary classes and methods for executing SQL operations from the C# environment.

The database connection is established through an Oracle connection string containing the required authentication and network parameters. The system uses the STMS database user, along with the local host address, Oracle listener port, and XE service name. The connection string is structured as follows:
User Id=stms;Password=tiger;Data Source=127.0.0.1:1521/XE;

The OracleConnection class is used to create and establish the database connection. When a database operation is required, the application invokes the Open() method to establish an active connection with the Oracle database. SQL operations are then executed through the OracleCommand class. Parameterized queries are implemented to securely process user-provided values, such as usernames, passwords, and mobile numbers, while reducing the risk of SQL injection.

The database connection mechanism supports major system operations, including **user registration and authentication**. During registration, user information is inserted into the Users table, where the UserId is automatically generated using an Oracle sequence. The generated UserId is subsequently used to store the corresponding mobile number in the UserPhone table. During authentication, the system verifies the submitted username and password and retrieves the corresponding RoleId. The retrieved role determines the user's access level within the system, such as **Admin (RoleId 1), Organizer (RoleId 2), Captain (RoleId 3), and Viewer (RoleId 5)**.

To maintain database consistency, related registration operations are executed within a database transaction. If all operations are completed successfully, the transaction is committed; otherwise, it is rolled back. Furthermore, exception handling is implemented to manage connection failures and SQL execution errors appropriately. Database resources are released after each operation using structured resource management. Overall, this database connectivity mechanism provides a reliable and controlled interface between the application layer and the Oracle database, supporting secure data access, transaction integrity, and role-based functionality.

Necessary codes

App.Config

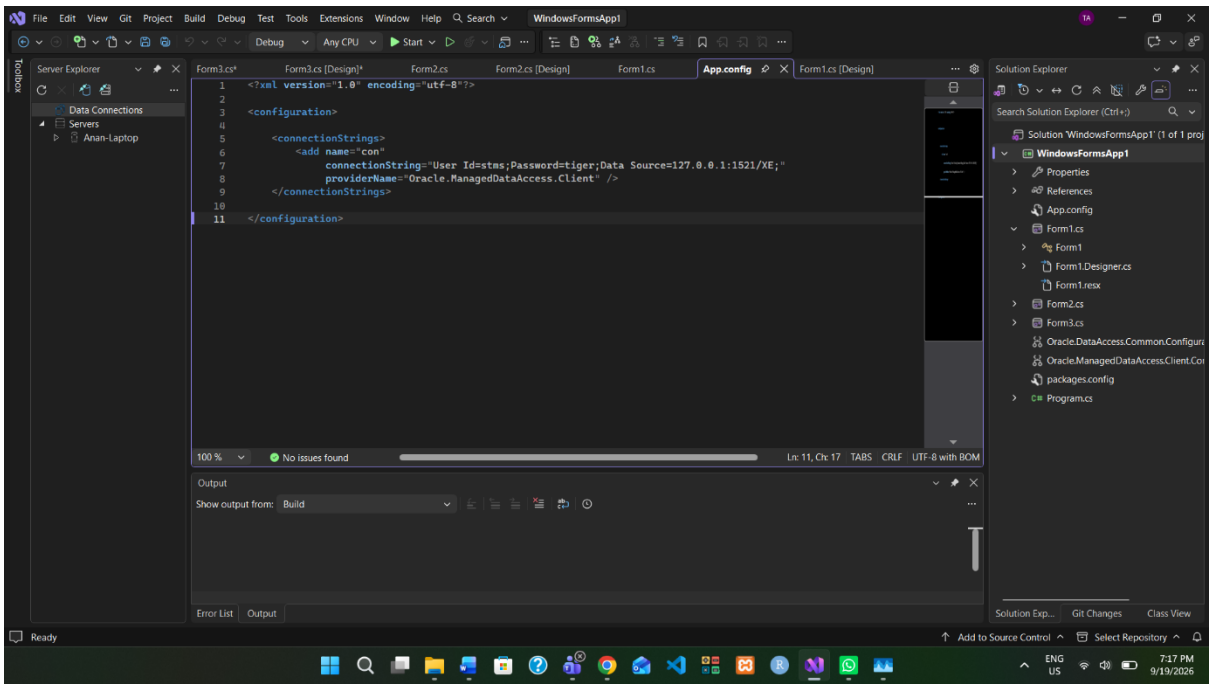
```
<?xml version="1.0" encoding="utf-8"?>

<configuration>

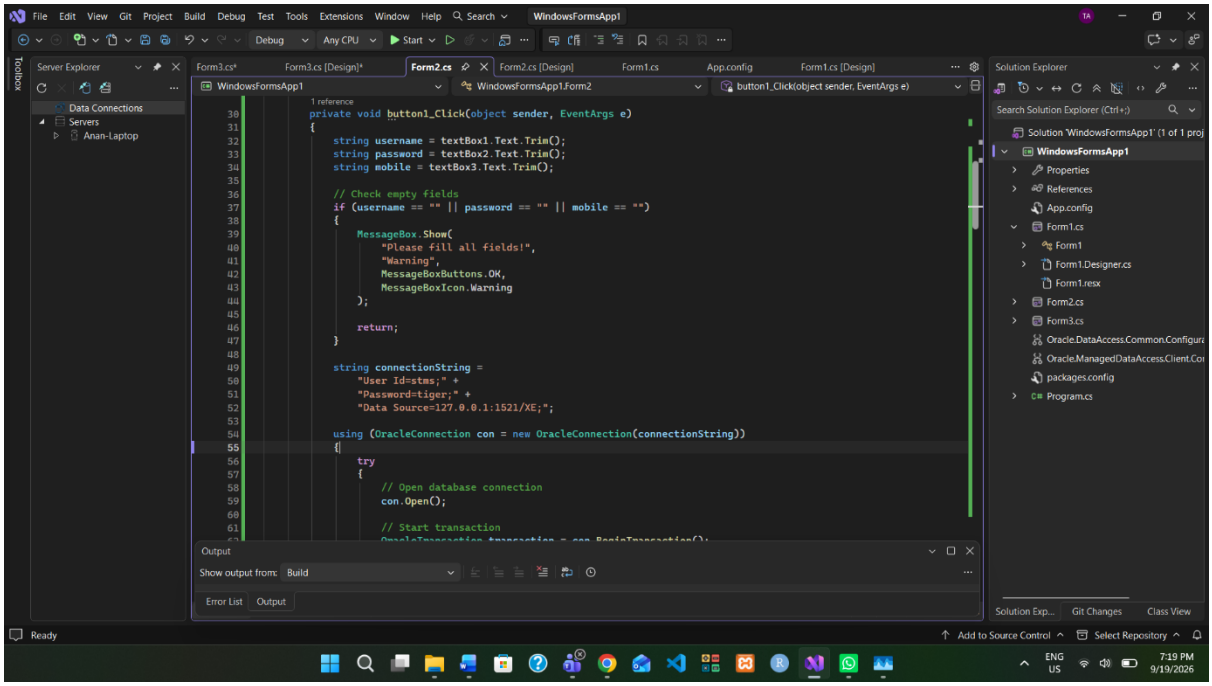
	<connectionStrings>
		<add name="con"
			connectionString="User Id=stms;Password=tiger;Data Source=127.0.0.1:1521/XE;"
			providerName="Oracle.ManagedDataAccess.Client" />
```

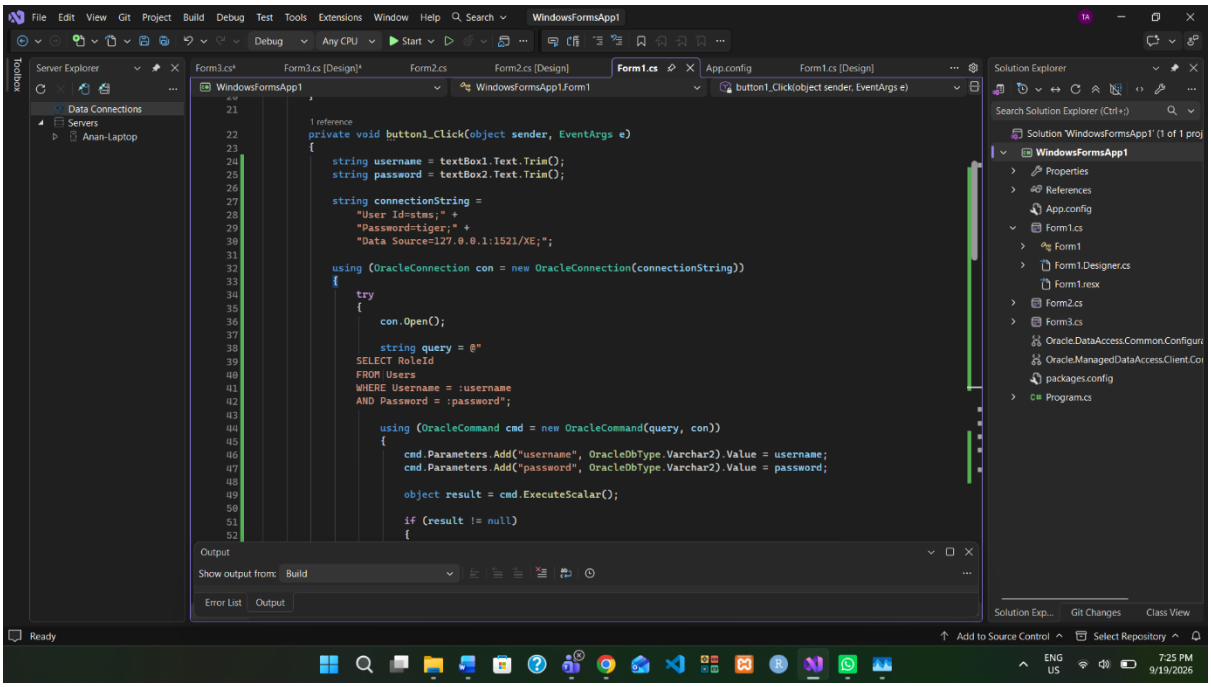
```
    </connectionStrings>

</configuration>
```



Login & regitration




File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
WindowsFormsApp1
Debug
Any CPU
Start
Server Explorer
Data Connections
Servers
Anan-Laptop
Form3.cs*
Form3.cs [Design]*
Form2.cs
Form2.cs [Design]
Form1.cs
App.config
Form1.cs [Design]
WindowsFormsApp1
WindowsFormsApp1.Form1
button1_Click(object sender, EventArgs e)
1 reference
private void button1_Click(object sender, EventArgs e)
{
string username = textBox1.Text.Trim();
string password = textBox2.Text.Trim();
string connectionString =
"User Id=stms;" +
"Password=tiger;" +
"Data Source=127.0.0.1:1521/XE;";
using (OracleConnection con = new OracleConnection(connectionString))
{
try
{
con.Open();
string query = @"
SELECT RoleId
FROM Users
WHERE Username = :username
AND Password = :password";
using (OracleCommand cmd = new OracleCommand(query, con))
{
cmd.Parameters.Add("username", OracleDbType.Varchar2).Value = username;
cmd.Parameters.Add("password", OracleDbType.Varchar2).Value = password;
object result = cmd.ExecuteScalar();
if (result != null)
{
Output
Show output from: Build
Error List
Output
Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'WindowsFormsApp1' (1 of 1 proj
WindowsFormsApp1
Properties
References
App.config
Form1.cs
Form1
Form1.Designer.cs
Form1.resx
Form2.cs
Form3.cs
Oracle.DataAccess.Common.Configur
Oracle.ManagedDataAccess.Client.Co
packages.config
Program.cs
Solution Exp...
Git Changes
Class View
Ready
Add to Source Control
Select Repository
ENG
US
7:25 PM
9/19/2026

THANK YOU